San Jose State University
CS 147 - Computer Architecture
Class Project - Phase 2

Overall Timeline and Grade Distribution

Due Date	Project Component	
2/26/26	Design Review	(5% of project grade)
3/10/26	Phase #1	(15% of project grade)
4/7/26	Phase #2	(30% of project grade)
4/16/26	Phase #2.1	(10% of project grade)
N/A	Phase #2.2	(Optional)
4/23/26	Phase #2.3 Caching	(10% of project grade)
5/7/26	Phase #3	(30% of project grade)

Phase #2 — Pipelined Design with Perfect Memory (30% of project grade)

At this point, the pipelined version of your design needs to be running correctly, but no optimizations are needed yet. Correctly means that it must detect and do the right thing on pipeline hazards (e.g., stall). You will still use the single-cycle memory model. We will follow similar protocol as demo1. We will run your tests and ask students with any failures to sign up for a demo with us.

Required student_custom tests

You must write at least two additional custom tests for pipelining and include them in the student custom test flow. Specifically, place at least two test programs in:

```
./assignments/project/testprograms/student_custom/
```

and add them to:

```
./assignments/project/testprograms/student_custom/all_phase_2.list
```

Writing more tests is strongly encouraged and will simplify debugging. At minimum, the presence of at least two student custom tests is required. The presence of a minimum of two (passing) student custom tests will be factored into the Phase 2 grade.

Instruction timeline

You must create and submit a document which should explain the behavior of your processor for the `perf-test-dep-ldst.asm` test. Please use the following format:

Cycle	Instruction Retired	Reason
1		
2		
⋮	⋮	⋮
Etc.		

The instruction retired (we will discuss what retired means in class, but essentially for our 5-stage in-order processor it means what is in Write Back that cycle) would either be one of the instructions from the test program or a "NOP" if dependencies necessitate any stall cycles. The reason column would explain why a stall was needed in that instance.

Please include this information in a PDF file titled `instruction_timeline.pdf`, located at:

```
./demo2/writeup/instruction_timeline.pdf
```

Synthesis

You should also synthesize your processor and submit the results of synthesis, including the area, cell, and timing reports. This is similar to what you did for Lab 3.

Submitting

To generate your Phase 2 submission file, use:

```
./run build_submission project phase_2
```

This follows the same submission workflow used in Phase 1: run the Phase 2 test flow, package the expected files, and produce a submission zip for upload.

Your generated submission zip will be written to:

```
./generated_turnins/project_phase_2
```

Upload the generated zip file to the corresponding Phase 2 Gradescope assignment.

Testing Commands

Base command form:

```
./run test [-v] project phase_2 [subtest]
```

Follow-on command form:

```
./run test [-v] project phase_2_x [-only] [-direct|-associative]
```

`x` is the follow-on phase to be tested. `-only` runs only that follow-on category plus `student_custom`. `-direct` and `-associative` are valid for `phase_2_3` and select a single cache implementation to test.

Required Test Categories

The base `phase_2` grading pass evaluates:

1. Your student custom tests (`student_custom`)
2. Simple instruction tests (`inst_tests`)
3. Complex tests from demo1 (`complex_demo1`)
4. Random tests for demo1:
   (a) `rand_simple`
   (b) `rand_complex`
   (c) `rand_ctrl`
   (d) `rand_mem`
5. Random/complex tests for demo2 (`complex_demo2`)

Additional categories are included when you run `phase_2_x` commands, as described in the Testing Commands section above.

The initial Phase 2 grading pass is based on `phase_2` categories. The 2.1/2.2/2.3 Follow-on phases are exercised by their corresponding `phase_2_x` commands.

A brief note of caution: if your `all_phase_2.list` has > 100 failures, the test runner will stop running tests from that list — please make sure to check your logs if you have many failures.

Running all tests can take significant time. Plan ahead and avoid waiting until the deadline window.

Follow-on phases

The next phases are intended as incremental extensions to your Phase 2 pipeline implementation. Phase #2.1 and the Cache Demo (#2.3) are required project milestones. Phase #2.2 is optional. Make sure your design passes the expected tests at each stage.

Phase #2.1 — Pipelined Design with Aligned Memory (10% of project grade)

This milestone is required.

At this step, replace the original single-cycle memory with the Aligned single-cycle memory. This is a very similar module, but it has an `err` output that is generated on unaligned memory accesses. Your processor should halt when an error occurs. Verify your design.

If you want to test, you can test with the `unaligned.list` test set:

```
./run test project phase_2_1 -only
```

This runs the Phase 2.1 category (which corresponds to the unaligned-access set) plus `student_custom`.

Aligned single-cycle memory specification (summary)

Before building your cache, you should use this memory to update and test your processor's interface to properly handle unaligned accesses. Many processors (e.g., MIPS) are byte addressable, but require that all accesses be aligned to their natural size (i.e., byte loads and stores can access any individual byte, but word loads and stores must access aligned words).

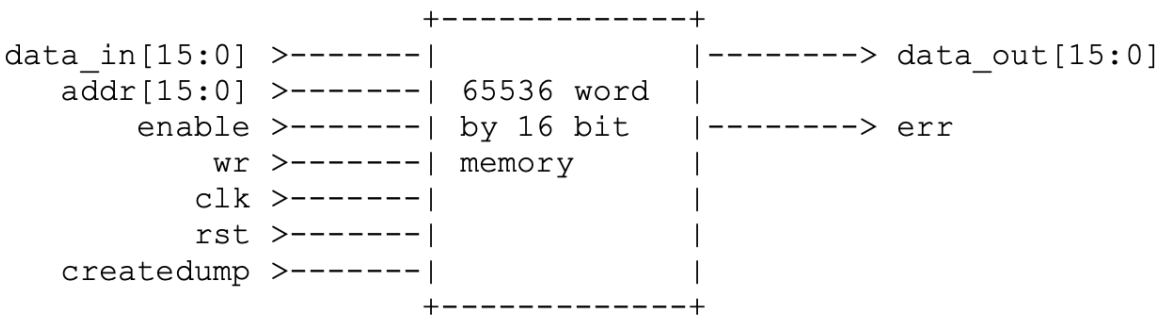
Since your processor only has word loads and stores, this is pretty simple. (To support byte stores, the memory would need byte write enable signals; to support byte loads, either the memory or the processor needs a mux to select the right byte.)

Notice that the memory always returns aligned data even on a misaligned load, and memory does not store anything on a misaligned store.

The Verilog source is provided at:

```
./demo2/verilog/phase2_1/memory2c_align.v
```

Since your single-cycle design must fetch instructions as well as read or write data in the same cycle, you will want to use two instances of this memory — one for data, and one for instructions.



During each cycle, the "enable" and "wr" inputs determine what function the memory will perform. On an unaligned access err is set.

enable	wr	Function	data_out	err
0	X	No operation	0	0
1	.0	Read	M[addr]	0
1	1	Write	Write data_in	0
1	X	X	if (addr[0]) set	1

Phase #2.2 — Pipelined Design with Stalling Memory (Optional)

No submission required. This is optional.

At this step, replace the single-cycle memory with the Stalling memory. This is a very similar module, but has `stall` and `done` signals similar to the cache you will build. Your pipeline will need to stall to handle these conditions. Verify your design.

- **Instruction memory:** First replace your instruction memory module with this stalling memory; keep your data memory module the same (i.e., aligned perfect memory from previous step). Verify your design. This will be easier to debug, as only one module's behavior has changed.
- **Data memory:** Now, replace your data memory module alone with this stalling memory; revert your instruction memory module back to the aligned perfect memory. Verify your design. This will be easier to debug, as only one module's behavior has changed.
- **Instruction and Data memory:** Now change both instruction and data memories to the stalling memory design. Verify your design.

Stalling memory specification (seed control)

This module has an interface identical to the cache interface in `mem_system_hier.v`.

Examining the source file `./demo2/verilog/phase2_2/stallmem.v`, you will see `rand_pat`, a shift register which controls the `ready` output. This is a random 32-bit number. You can change its value by changing the seed used for random number generation.

To run the stalling-memory category:

```
./run test project phase_2_2 -only
```

For seed-specific debugging, use interactive simulation and/or adjust the seed initialization in `./demo2/verilog/phase2_2/stallmem.v`.

Phase #2.3 — Cache Demo: Working Cache Memory System (10% of project grade)

In this phase you will implement a cache-based memory system to replace the idealized memory used earlier in the project. Your final processor will use both an instruction cache and a data cache. For this stage of the project you will design and verify the cache subsystem that will ultimately be integrated into your processor design.

You will first implement and test a **direct-mapped cache**. After that design is working and verified, you will extend it to a **two-way set associative cache**.

The cache storage arrays and the main memory module are already provided. Your task is to implement the **memory system controller** that coordinates the cache and memory modules.

Project Files

The cache files for this stage are organized under:

```
./demo2/verilog/phase2_3/
```

Each cache implementation directory contains a number of modules.

File	Description
cache.v	Cache storage structures (data, tag, valid, dirty)
clkrst.v	Clock and reset module
final_memory.v	Memory bank implementation
four_bank_mem.v	Four-banked memory wrapper
mem.addr	Example address trace file
memc.v	Data storage module used by cache
memv.v	Valid-bit storage module used by cache
mem_system_hier.v	Top-level wrapper used for testing
mem_system_perfbench.v	Trace-based testbench
mem_system_randbench.v	Random testbench
mem_system_ref.v	Reference implementation used by testbench
mem_system.v	Memory system module **(this is the file you modify)**
.syn.v	Synthesizable memory modules

In most cases, `mem_system.v` is the only module that must be modified.

Four-Banked Memory

The provided memory module models a modern memory system by dividing memory into four independent banks.

- Memory is **64 KB**
- Word size is **16 bits**
- Memory is **byte addressable but word aligned**
- Memory is split into **four banks**

The least significant address bits determine the memory bank. Because addresses are word aligned, bit 0 is always zero. Therefore bits `Addr[2:1]` determine the bank.

If two accesses target the same bank within four cycles, the second request will stall.

Key signals:

Signal	Direction	Width	Description
Addr	In	16	Address of the memory operation
DataIn	In	16	Data written during write operations
wr	In	1	Write enable signal
rd	In	1	Read enable signal
clk	In	1	Clock
rst	In	1	Reset
createdump	In	1	Dump memory contents to files
DataOut	Out	16	Data returned on reads
stall	Out	1	Indicates bank conflict
Busy	Out	4	Busy status for each bank
err	Out	1	Raised on unaligned accesses

Cache Organization

The cache contains **256 lines**. Each line contains:

- Valid bit
- Dirty bit
- 5-bit tag
- Four 16-bit words

Cache interface signals:

Signal	Direction	Width	Description
enable	In	1	Enables cache access
index	In	8	Cache index
offset	In	3	Selects word within cache line
comp	In	1	Enables tag comparison
write	In	1	Write enable
tag_in	In	5	Tag value from processor
data_in	In	16	Data written to cache
valid_in	In	1	Valid bit written during fills
clk	In	1	Clock
rst	In	1	Reset
createdump	In	1	Dump cache contents
hit	Out	1	Tag match indicator
dirty	Out	1	Dirty bit state
tag_out	Out	5	Tag read during eviction
data_out	Out	16	Data read from cache
valid	Out	1	Valid bit state
err	Out	1	Error condition

Cache Access Modes

The cache operates using two control signals: `comp` and `write`. The four possible combinations are:

Compare Read (comp=1, write=0)

Used during load instructions.

- Tag is compared with stored tag
- If hit, data is returned immediately
- If miss, the valid and dirty bits of the current line are observed

Compare Write (comp=1, write=1)

Used during store instructions.

- If hit, data is written to the cache line
- Dirty bit is set
- If miss, cache contents are not modified

Access Read (comp=0, write=0)

Used internally when evicting a cache line.

- Tag, valid, dirty, and data values are read from the cache

Access Write (comp=0, write=1)

Used when filling a cache line from memory.

- Data and tag are written into the cache
- Valid bit is set
- Dirty bit is cleared

State Machine

You must design a state machine to control cache behavior. Either a Mealy or Moore machine may be used (though Moore designs are typically easier).

Your controller must manage:

- Cache hit detection
- Cache misses
- Memory read operations
- Dirty-line writebacks
- Cache line fills
- Stall and done signaling

A state diagram for your cache controller must be submitted as part of this phase.

Direct-Mapped Cache

First implement a direct-mapped cache in the directory:

```
./demo2/verilog/phase2_3/cache_direct/
```

You should thoroughly test this version before proceeding to the associative version.

Testing

Use the project test wrapper for this follow-on phase:

```
./run test project phase_2_3
```

By default, this runs **both** cache implementations (direct-mapped and two-way associative).

Run only the 2.3 category (plus `student_custom`):

```
./run test project phase_2_3 -only
```

Test only one implementation:

```
./run test project phase_2_3 -direct
```

```
./run test project phase_2_3 -associative
```

Verbose mode / targeted debugging:

```
./run test -v project phase_2_3 [subtest]
```

Two-Way Set-Associative Cache

After verifying the direct-mapped cache, extend your design to a two-way set associative cache in directory:

```
./demo2/verilog/phase2_3/cache_assoc/
```

Two cache modules are instantiated, and hit detection is performed across both ways.

Replacement Policy

To ensure deterministic grading, you must implement the following pseudo-random replacement policy.

1. Maintain a flip-flop named `victimway`.
2. Initialize `victimway` to zero.
3. On each cache read or write, invert `victimway`.
4. If both cache ways are valid during replacement, select the way indicated by `victimway`.
5. If one way is invalid, install in the invalid way.

Submission

Use the standard submission wrapper:

```
./run build_submission project phase_2
```

The generated submission zip is written to:

```
./generated_turnins/project_phase_2/
```

Your Phase 2.3 work must be present in:

```
./demo2/verilog/phase2_3/cache_direct/
./demo2/verilog/phase2_3/cache_assoc/
```

Required artifacts include:

- Your cache controller state diagram (`state_machine.pdf`)
- All Verilog modules
- Any additional files required by your Phase 2.3 implementation/testing flow

Place `state_machine.pdf` in:

```
./demo2/writeup/state_machine.pdf
```